



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 • Design & Analysis of Algorithms • Fall 2026

Big-O — The Definition, Ω and Θ

What the notation asserts, how to prove it, and the four ways it gets misused

Session 4 of 24 • Tuesday 29 September 2026 • 2:30 – 3:50 PM

• Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 4 OF 24

Big-O — The Definition, Ω and Θ

Tuesday, 29 September 2026 • 2:30 – 3:50 PM

LEARNING OBJECTIVES

- ✓ State the formal definition of O and explain what each part of it does.
- ✓ Prove a Big- O claim by exhibiting a witness pair (c, n_0) .
- ✓ Define Ω and Θ and say which one a given claim requires.
- ✓ Identify and correct the four most common misuses of the notation.

FROM SESSION 3

Three moves — drop constants, keep the dominant term, think large n — turn a counted running time into a growth class. Today we find out what that notation *means*, precisely enough to argue with.

1 • The Definition 25 min

DEFINITION • BIG-O

$$f(n) = O(g(n))$$

if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

In words: **beyond some point, f never exceeds g by more than a fixed factor.**

Every piece of that sentence is doing a job, and each corresponds to one of Session 3's moves.

The constant c — Move 1

We are allowed to scale g upward as much as we like. That is exactly “drop the constants” made formal: $5n$ and n differ by a factor we are permitted to absorb into c .

The threshold n_0 — Move 3

The inequality need only hold *eventually*. Small inputs are exempt, which is why $3n^2 + 17n + 42$ can be $O(n^2)$ even though at $n = 1$ it is twenty times larger than n^2 .

The direction $\leq$ — a ceiling

O gives an upper bound and nothing more. It is a promise that things will be no worse than this.

Move 2 is a consequence

We do not need a separate rule for dropping lower-order terms: they are absorbed by c and n_0 automatically. Try proving $n^2 + 1000n = O(n^2)$ and watch it happen.

1.1 A proof, in three lines

The definition is an existence claim, so proving one means **exhibiting a witness**: a pair (c, n_0) that works.

There is no cleverness required, and no “best” witness to find.

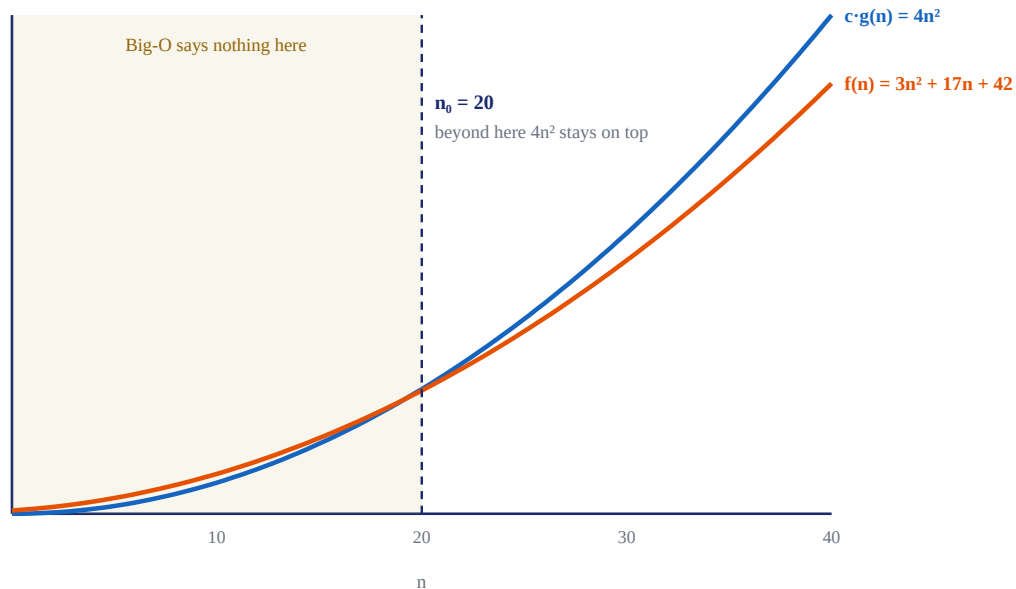
CLAIM • $3N^2 + 17N + 42 = O(N^2)$

Try $c = 4$. We need $3n^2 + 17n + 42 \leq 4n^2$ for all large n .

Rearranged, that is $n^2 - 17n - 42 \geq 0$.

The positive root of $n^2 - 17n - 42$ is $(17 + \sqrt{457})/2 \approx 19.2$, and the parabola opens upward, so the expression is non-negative for every $n \geq 20$.

Witness: $c = 4$, $n_0 = 20$. $\square$



What the proof says, drawn. The orange curve is $f(n)$; the blue curve is $4n^2$. To the left of $n_0 = 20$ the orange curve is on top and the claim would be false — which is exactly why the definition excuses that region. From n_0 onward the blue curve stays above, for ever, and that is the whole content of “ f is $O(n^2)$ ”.

Any valid witness will do

$c = 100$, $n_0 = 1$ also works for this claim, and is easier to verify. You are never asked to find the tightest witness — only to produce one. This is why Big-O proofs are usually short, and why a proof that is getting long usually means the claim is false.

1.2 Two more, quickly

CLAIM · $5N + 300 = O(N)$

Try $c = 6$: we need $5n + 300 \leq 6n$, i.e. $300 \leq n$.

Witness: $c = 6$, $n_0 = 300$. □

CLAIM · N IS $O(N^2)$ — A TRUE BUT WEAK STATEMENT

We need $n \leq c \cdot n^2$, which holds with $c = 1$ for every $n \geq 1$.

Witness: $c = 1$, $n_0 = 1$. □

Note what this means: a linear algorithm **is** $O(n^2)$. The statement is not wrong, merely uninformative — a ceiling far above the roof.

Big-O only bounds from above. That last example shows why we need more: if every linear algorithm is also $O(n^2)$, then O alone cannot express “this really is quadratic”. Two companions complete the picture.

DEFINITIONS · THE THREE BOUNDS

$f(n) = O(g(n)) \quad \exists c, n_0 : f(n) \leq c \cdot g(n) \text{ for } n \geq n_0$ — a ceiling

$f(n) = \Omega(g(n)) \quad \exists c, n_0 : f(n) \geq c \cdot g(n) \text{ for } n \geq n_0$ — a floor

$f(n) = \Theta(g(n))$ both at once — f is squeezed between two multiples of g

O — “no worse than”

A promise to your user.
Useful when you know a
limit on the damage but not
the exact behaviour.

Ω — “no better than”

A limit on what any
improvement could achieve.
Used for impossibility results:
every comparison-based
sorting algorithm is $\Omega(n \log n)$,
so no one will ever write a
comparison sort that is linear.

Θ — “exactly this shape”

The honest answer whenever
you know it. Linear search's
worst case is $\Theta(n)$: it is at
most linear and at least linear.

Statement	True?	Why
$3n + 5$ is $O(n)$	yes	$c = 4, n_0 = 5$
$3n + 5$ is $O(n^2)$	yes	true but weak — a ceiling far too high
$3n + 5$ is $\Theta(n^2)$	no	it is not $\Omega(n^2)$: $3n + 5$ eventually falls far below any $c \cdot n^2$
n^2 is $\Omega(n \log n)$	yes	n^2 grows at least as fast
2^{n+1} is $\Theta(2^n)$	yes	$2^{n+1} = 2 \cdot 2^n$, a constant factor
2^{2n} is $\Theta(2^n)$	no	$2^{2n} = (2^n)^2$ — that is a squaring, not a factor

THE LAST TWO ROWS ARE THE EXAM QUESTION

An exponent added inside is a constant factor; an exponent multiplied inside is a squaring. Students lose marks on this every year. Test it: at $n = 10$, $2^{n+1} = 2048$ and $2^n = 1024$ — a factor of 2. But $2^{2n} = 1,048,576$ — a factor of 1024, and that factor grows with n .

Why everybody writes O when they mean Θ

Convention and laziness, in that order. It is harmless when the bound happens to be tight and misleading when it is not. The rule in this course: when you know the shape exactly, write Θ and be right; when you only have a ceiling, write O and say that is what you have.

3 · Four Misreadings 20 min

“ $O(n)$ means it takes n steps”

No. It means the cost is bounded above by *some multiple* of n , eventually. An $O(n)$ function may take $3n + 7$ steps, or $0.5n$ steps, or on some inputs a single step.

“ O is the worst case, Ω is the best case”

This is the most damaging error in the topic, and it is simply false. **Case** and **bound** are independent. The case selects which inputs you look at; the bound describes how that cost grows. You can say “the best case is $\Theta(1)$ ” or “the worst case is $\Omega(n)$ ” and both are meaningful.

“A faster program disproves an $O(n^2)$ claim”

It does not. A linear algorithm *is* $O(n^2)$. To refute an upper bound you must exhibit an input on which the function is *slower* than the bound — upper bounds are refuted from above only.

“Big- O tells me which program to use”

Only for large inputs. At $n = 20$ a quadratic routine with a small constant routinely beats a linearithmic one with a large constant. Big- O tells you what happens when n grows — which matters precisely because you are usually wrong about how big n will get.

CASE AND BOUND, ONCE MORE

Linear search: **best case $\Theta(1)$** (target at index 0), **worst case $\Theta(n)$** (target absent), **average case $\Theta(n)$** (about $n/2$ comparisons, and $n/2$ is $\Theta(n)$). Three cases, each with its own exact bound. Unless we say otherwise, every claim in this course is about the **worst case** — the only one that is a promise rather than a hope.

“ f is $O(g)$ ” asserts that beyond some threshold, f stays below a fixed multiple of g . Prove it by naming c and n_0 ; no cleverness is required and any valid pair will do. O is a ceiling, Ω a floor, Θ both — and the case you analyse is a separate question from the bound you use.

IN-CLASS EXERCISES (work in pairs)

1. Prove from the definition, giving a witness: (a) $7n + 40 = O(n)$; (b) $n^2 + 1000n = O(n^2)$; (c) $n^2/100 = \Omega(n)$.
2. True or false, with a reason: (a) $n \log n$ is $O(n^2)$; (b) n^2 is $O(n \log n)$; (c) $3n + 5$ is $\Theta(n)$; (d) every function is $O(2^n)$.
3. Show that n^2 is *not* $O(n)$. (Hint: suppose a witness existed and derive a contradiction by choosing n large enough.)
4. A student writes: “insertion sort is $O(n)$ because on sorted input it does one pass.” Identify precisely which of the four misreadings this is, and rewrite the statement correctly.
5. Explain why $\log_2 n$ and $\log_{10} n$ are the same up to a constant factor, and what that means for writing $O(\log n)$ without a base.

Homework • due Friday 2 October

1. For each of these, decide whether it is O , Ω , Θ of n^2 , and justify every answer from the definition with an explicit witness or a contradiction: $5n^2 + 3n$; $n^2/1000$; $n \log n$; $n^2 \log n$; $4^{\log_2 n}$.
2. Write a short Python script that, for a given c and n_0 , checks numerically whether $3n^2 + 17n + 42 \leq c \cdot n^2$ for n from n_0 to 10,000. Use it to confirm that (4, 20) is a witness and that (3, anything) is not.
3. State the best, worst and average case of binary search, each with a Θ bound, and give an input achieving each.
4. Prove that if f is $O(g)$ and g is $O(h)$, then f is $O(h)$. (Two witnesses in, one witness out.)
5. Reading: CLRS §3.1–3.2. Half a page on why Θ is the statement we want and O the one we usually get.

Next session • Friday 2 October: Counting loop nests systematically. Sequences add, nests multiply, and dependent nests need a series — plus the library calls that hide a loop inside what looks like one innocent line.

